ATTENDANCE FLOW

Smart Attendance. Better Workforce.

Complete Software Documentation

Architecture • Installation • Database • Roles • Authentication • Biometrics • GPS • Attendance • Payments • Security • Troubleshooting

Document control	Value
Product	Attendance Flow
Publisher	Ablaz Technologies Limited
Technology baseline	PHP 8, MySQL, Bootstrap 5, vanilla JavaScript
Deployment targets	XAMPP local development; shared PHP hosting
Timezone policy	Store timestamps in UTC; display in Africa/Lagos
Currency	Nigerian Naira (■ / NGN)
Version	Documentation baseline 1.0
Status	Implementation-dependent details are explicitly identified

Copyright © Ablaz Technologies Limited. All rights reserved, subject to the license selected by the project owner.

Important accuracy note: the project brief lists 25 table names while describing the database as “21 tables.” No actual SQL schema, source code, configured provider credentials, or deployment environment was supplied. Therefore, this document distinguishes stated requirements from implementation details that must be verified in the codebase. Suggested column designs are reference designs, not proof that those columns already exist.

Contents

Section	Contents
1. README.md	Project overview, stack, quick start, folder map
2. docs/01-installation.md	Local setup and deployment configuration
3. docs/02-architecture.md	System architecture and tenancy
4. docs/03-database.md	Table inventory, reference schema, relationships
5. docs/04-roles-and-permissions.md	Permission matrix and data visibility
6. docs/05-authentication.md	Registration, login, verification, reset
7. docs/06-face-verification.md	Enrollment, embeddings, matching, limitations
8. docs/07-gps-geofencing.md	Location validation and Haversine
9. docs/08-attendance-workflow.md	Clock-in/out, manual attendance, reports
10. docs/09-subscriptions-and-payments.md	Plans, Paystack, manual payments, receipts
11. docs/10-security.md	Controls, threat model, operational security
12. docs/11-troubleshooting.md	Common faults and recovery

Cross-references are shown as relative Markdown paths to match the intended repository structure. This PDF combines all 12 files into one reference document.

1. README.md

Attendance Flow

Tagline: Smart Attendance. Better Workforce.

Attendance Flow is a multi-tenant employee attendance management system published by Ablaz Technologies Limited. Its intended workflow combines organization and department administration with employee clock-in/clock-out, face verification, GPS geofencing, attendance review, subscriptions, payment records, and support tickets. PHP and MySQL provide the server-side foundation, Bootstrap 5 provides responsive interface components, and vanilla JavaScript supports browser interactions. Actual readiness depends on the configured database schema, face engine, mail provider, payment credentials, and production security settings.

Badges

```
![PHP 8](https://img.shields.io/badge/PHP-8.x-777BB4)
![MySQL](https://img.shields.io/badge/Database-MySQL-4479A1)
![Bootstrap](https://img.shields.io/badge/UI-Bootstrap%205-7952B3)
![License](https://img.shields.io/badge/License-Project%20Owner%20Defined-lightgrey)
```

These are illustrative Markdown badge URLs; the badges do not certify tests, security audits, or release status.

Key features

• Organization registration with email verification.

• Five role types with server-side authorization and organization-level data isolation.

• Face enrollment and one-to-one verification using 128-dimensional embeddings where the selected engine supports that format.

• GPS geofencing with server-side Haversine distance calculation.

• Employee clock-in and clock-out requiring configured face and location checks.

• Duplicate attendance prevention, late detection, worked-hours calculation, and attendance history.

• Department-head manual attendance with a recorded reason.

• Attendance submission that locks submitted records for review, according to implemented workflow rules.

• Subscription plans, trials, Paystack checkout, manual payment records, and printable receipts.

• Support tickets with threaded messages; audit logging for sensitive actions.

• Platform settings for general configuration, email, payments, face API, attendance, security, and system health.

Technology stack

Layer	Technology / policy
Server	PHP 8.x; PDO for database access
Database	MySQL; schema must be imported from the project's actual SQL file
Frontend	HTML, CSS, Bootstrap 5, vanilla JavaScript
Biometrics	Selected face engine: browser ONNX, AWS, or Azure only if implemented and configured
Payments	Paystack integration plus manual bank-transfer review, if enabled
Local development	XAMPP
Production	HTTPS-enabled shared PHP hosting with supported PHP extensions
Time	Persist UTC timestamps; display using Africa/Lagos

Quick start (5 steps)

• Install XAMPP with a supported PHP 8 release and MySQL/MariaDB. Start Apache and MySQL.

• Copy the project into the web root, commonly `C:\xampp\htdocs\attendance-flow`.

• Create a dedicated database and import the SQL schema supplied with the actual project. Do not create production tables from the reference-only column suggestions in this document.

• Copy `.env.example` to `.env` if the application uses dotenv, then set database, base URL, mail, face-provider, and payment values. Never commit real secrets.

• Open the configured local URL, complete the setup/first-admin process documented by the application, and run the verification checklist in `docs/01-installation.md`.

Suggested folder structure

```
attendance-flow/
■■■ app/                # Application services, policies, validation
■■■ config/             # Non-secret configuration and bootstrap
■■■ database/           # Schema, migrations, seed data
■■■ docs/               # This documentation set
■■■ public/             # Web document root: index.php, assets
■   ■■■ assets/css/
■   ■■■ assets/js/
■   ■■■ assets/images/
■■■ resources/views/    # Templates / view fragments
■■■ routes/             # Route definitions, if the project uses them
■■■ storage/            # Logs, temporary files; never public
■■■ tests/              # Automated tests, if present
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

The tree is a recommended organization, not a claim that the existing repository already uses these exact paths.

Documentation index

File	Purpose
docs/01-installation.md	Install and configure locally / production
docs/02-architecture.md	Layers, request lifecycle, tenancy
docs/03-database.md	Table inventory and relationships
docs/04-roles-and-permissions.md	Roles, capabilities, restrictions
docs/05-authentication.md	Login, verification, password reset
docs/06-face-verification.md	Biometric enrollment and verification
docs/07-gps-geofencing.md	Location validation
docs/08-attendance-workflow.md	Attendance rules and reporting
docs/09-subscriptions-and-payments.md	Plans, checkout, receipts
docs/10-security.md	Threat model and controls
docs/11-troubleshooting.md	Operational troubleshooting

License and copyright

Copyright © Ablaz Technologies Limited. All rights reserved unless a separate license file grants other rights. Add a LICENSE file before distributing the code. Do not imply the project is open source without an explicit license.

2. docs/01-installation.md

Prerequisites

Requirement	Recommended baseline	Notes
Operating system	Windows for XAMPP; Linux hosting for production	Commands below are illustrative; adapt paths.
PHP	PHP 8.x supported by project	Check required extensions: PDO MySQL, OpenSSL, JSON, mbstring, fileinfo; sodium where available.
Database	MySQL / compatible MariaDB	Use a dedicated database and least-privilege account.
Web server	Apache locally; host-provided web server in production	HTTPS required for camera and geolocation in production.
Browser	Recent Chrome, Edge, Firefox, or Safari	Camera/GPS permissions and ONNX/WebAssembly support vary.
External services	SMTP, face engine, Paystack test/live credentials	Only configure providers actually implemented.

Local installation

• Download and install XAMPP from its official distribution source. Start Apache and MySQL from the control panel.

• Place the application directory under the XAMPP document root, for example `C:\xampp\htdocs\attendance-flow`. Avoid placing secrets or logs inside a publicly served directory.

• Open phpMyAdmin at the local XAMPP address, or use the MySQL CLI, and create a database with `utf8mb4` character encoding.

• Import the actual project schema file. If the repository has migrations, use the project's documented migration command instead of importing a second schema.

• Configure environment variables, verify the application base URL, and set write permissions only on required storage directories.

• Open the local application URL and complete the actual bootstrap or first-administrator procedure. Do not assume a default username or password unless the project explicitly documents one.

Create the database

```
CREATE DATABASE attendance_flow
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

-- Prefer a dedicated account instead of root for application access.
CREATE USER 'attendance_app'@'localhost'
  IDENTIFIED BY 'REPLACE_WITH_A_LONG_RANDOM_PASSWORD';

GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX
  ON attendance_flow.* TO 'attendance_app'@'localhost';

FLUSH PRIVILEGES;
```

Limit migration privileges after installation if the runtime application does not need schema changes. On shared hosting, use the database name, username, and host assigned by the hosting provider.

Import the schema

```
# Example only; replace the file name and credentials.
mysql -u attendance_app -p attendance_flow < database/schema.sql
```

Alternatively, use phpMyAdmin → select the database → Import → choose the SQL file. Verify import success and inspect the actual table list. The brief's inventory names 25 tables, despite calling it a 21-table database; reconcile that discrepancy against the SQL schema before release.

Environment configuration

```
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost/attendance-flow/public
APP_TIMEZONE=Africa/Lagos

DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=attendance_flow
DB_USERNAME=attendance_app
DB_PASSWORD=REPLACE_WITH_A_LONG_RANDOM_PASSWORD

MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=replace-me
MAIL_PASSWORD=replace-me
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=no-reply@example.com

FACE_ENGINE=browser_onnx
FACE_API_URL=
FACE_API_KEY=

PAYSTACK_PUBLIC_KEY=
PAYSTACK_SECRET_KEY=
PAYSTACK_WEBHOOK_SECRET=
```

This is an illustrative environment template, not a guarantee that the application reads these exact variable names. Confirm the names in its configuration/bootstrap code. Set `APP_DEBUG=false` in production. Keep `.env` outside version control and deny direct web access to it.

First login and verification checklist

• Confirm the homepage loads without PHP warnings or stack traces.

• Confirm the database connection works and the application uses the intended database.

• Create or activate the first System Administrator through the supported setup process.

• Verify email delivery using a non-sensitive test account.

• Test login, logout, invalid credentials, rate limiting, and password reset.

• Confirm each role is denied access to pages and API endpoints outside its permissions.

• Test organization isolation using two separate test organizations.

• Test face enrollment and verification only after the selected face engine is configured.

• Test geofence acceptance and rejection at known distances from a configured work location.

• Test clock-in, clock-out, duplicate prevention, late detection, submission locking, and reports.

• Use Paystack test mode before any live transaction; test webhook signature validation and idempotency.

• Confirm HTTPS, secure cookies, backups, logs, and production debug settings before launch.

Production deployment checklist

• Set the web document root to the intended public directory, if the project structure supports it.

• Use HTTPS and secure session cookies; camera and geolocation browser APIs generally require a secure context.

• Store credentials in host environment configuration or a protected file outside the document root.

• Disable verbose errors and directory listing; keep backups outside public paths.

• Confirm PHP extensions, upload limits, request timeouts, cron needs, and outbound SMTP/API connectivity.

• Run security, tenancy, payment, and recovery tests on staging before production.

3. docs/02-architecture.md

System diagram

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
flowchart TB
  E[Employee browser / phone] --> W[HTTPS Web Interface]
  A[Organization / Department / Platform staff] --> W
  W --> C[PHP Controllers / Request Handlers]
  C --> AUTH[Authentication and Authorization]
  C --> SVC[Application Services]
  SVC --> DB[(MySQL)]
  SVC --> FACE[Face engine adapter]
  SVC --> GEO[Server-side geofence validator]
  SVC --> PAY[Paystack / manual payment workflow]
  SVC --> MAIL[SMTP email service]
  SVC --> AUD[Audit and operational logs]
```

Layer responsibilities

Layer	Responsibilities	Security boundary
Browser / presentation	Forms, camera capture, geolocation prompt, status display, responsive UI	Treat all browser values as untrusted; never authorize in JavaScript alone.
Request handling	Parse requests, validate input shape, call services, render responses	Enforce method, CSRF, session and authorization checks.
Application services	Registration, attendance state changes, subscription and ticket workflows	Apply business rules consistently and transactionally.
Domain/security policies	Role permissions, tenant scope, attendance and subscription rules	Deny by default; verify every request and object scope.
Data access	PDO prepared statements, transactions, repositories/queries	Use parameter binding and organization-scoped predicates.
External adapters	Face APIs/models, email provider, Paystack	Validate responses, time out safely, protect credentials, log outcomes.
Operations	Backups, monitoring, audit logs, configuration	Restrict access and avoid logging secrets or biometric vectors.

Multi-tenancy strategy

The stated design is multi-tenant: an organization’s employees, departments, attendance, locations, and subscription data must not leak to another organization. A shared database is a common implementation, but the actual schema must confirm how tenant keys are represented.

• Resolve the current organization from the authenticated account and trusted server-side relationships—not from a client-supplied organization ID alone.

• Scope every organization-owned query using the authenticated tenant key. Apply the same rule to reads, updates, deletes, exports, attachments, and background jobs.

• Use foreign keys where feasible and composite unique constraints where they enforce tenant-local uniqueness, such as `(organization_id, employee_number)`.

• Test cross-tenant access attempts using guessed IDs and modified URLs. A hidden menu item is not authorization.

• Keep platform-wide System Administrator actions separate from organization-scoped actions and audit them.

Request lifecycle

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
sequenceDiagram
  participant B as Browser
  participant P as PHP request handler
  participant A as Auth/Policy layer
  participant S as Application service
  participant D as MySQL
  B->>P: HTTPS request + session + CSRF (for POST)
  P->>A: Validate session, role, tenant scope
```

```
A-->>P: Allow or deny
P->>S: Validated command
S->>D: Prepared query / transaction
D-->>S: Result
S-->>P: Domain outcome
P-->>B: Escaped HTML / JSON response
```

Directory responsibilities

Directory	Purpose
`public/`	Only files intended to be web-accessible: front controller, CSS, JavaScript, public images.
`app/`	Business services, authorization policies, validators, data access and integration adapters.
`config/`	Application configuration and bootstrap; secrets should come from protected environment configuration.
`database/`	Canonical schema, migrations, seeders, and test fixtures.
`resources/views/`	Server-rendered templates and reusable UI components.
`routes/`	Route definitions where the application uses a routing layer.
`storage/`	Non-public logs, temporary files, generated receipts and other protected artifacts.
`tests/`	Unit, integration, tenancy, authorization, and payment tests.
`docs/`	Markdown source documentation.

See [Installation](01-installation.md), [Database](03-database.md), and [Security](10-security.md) for operational details.

4. docs/03-database.md

Schema accuracy warning: the brief provides table names but not actual column definitions or SQL. The following is a reference design for documentation and review, not a substitute for the real schema. Before publishing a schema contract, inspect the migration/SQL files and replace the suggested columns and types with exact definitions.

Table inventory

Table	Purpose / key design note
organizations	Organization tenant profile; identity, status, contact and timestamps.
users	Login identities, role, organization linkage, password hash and account status.
organization_settings	Per-organization preferences and policy configuration.
departments	Departments belonging to an organization.
employees	Employee profile and employment identifiers linked to a department and login identity as designed.
face_profiles	Encrypted biometric embedding/profile, model metadata and enrollment state.
face_verifications	Verification attempts, outcome, score, engine and timestamps; avoid storing unnecessary biometric data.
work_locations	Organization-owned named locations, coordinates, allowed radius and status.
attendance	Clock-in/out timestamps, source, status, location/verification references, late/worked-hour fields.
attendance_submissions	Department/period submission, submitter, status, lock/review timestamps.
subscription_plans	Plan code, price in NGN, limits, feature flags and billing interval.
subscriptions	Organization subscription state, plan, trial/billing periods and provider references.
subscription_history	Append-only history of subscription state changes.
subscription_requests	Requests for plan changes, renewals or manual payment review.
payment_records	Payment amount, currency, provider reference, state and idempotency data.
payment_receipts	Receipt number, payment reference, amount, issue time and generated document metadata.
email_verifications	Hashed verification token, expiry, used time and user reference.
password_resets	Hashed reset token, expiry, used time and user reference.
login_attempts	Account/IP attempt metadata and time window for rate limiting.
support_tickets	Ticket owner, organization, subject, priority, status and timestamps.
support_messages	Threaded ticket messages, sender and timestamp.
notifications	Recipient, message metadata, read status and timestamp.
audit_logs	Actor, action, target, outcome, request metadata and timestamp.
email_logs	Delivery attempt status, provider message ID and error category; exclude secrets.
system_settings	Platform-wide configuration values, with sensitive values stored encrypted or externally.

Count check: this inventory contains 25 table names. The brief says “21 tables,” so the discrepancy must be resolved against the actual schema before deployment.

Reference column patterns

Use these as a review checklist. Exact columns, SQL types, nullability, defaults, indexes, and constraints must be taken from the implementation.

Table group	Illustrative columns and types	Constraints / notes
organizations	id BIGINT; name VARCHAR; slug VARCHAR; status VARCHAR; created_at DATETIME	Primary key; unique slug if used; UTC timestamps.

Table group	Illustrative columns and types	Constraints / notes
users	id BIGINT; organization_id BIGINT NULL; email VARCHAR; password_hash VARCHAR; role VARCHAR; status VARCHAR	Unique email policy must be explicit; FK to organization where applicable.
departments	id BIGINT; organization_id BIGINT; name VARCHAR; department_head_user_id BIGINT NULL	FK to organization; tenant-local unique name if desired.
employees	id BIGINT; organization_id BIGINT; department_id BIGINT; user_id BIGINT NULL; employee_number VARCHAR; status VARCHAR	FKs and tenant-local uniqueness; avoid mixing login identity and employment profile without clear rules.
face_profiles	id BIGINT; employee_id BIGINT; embedding_ciphertext BLOB/TEXT; model_version VARCHAR; enrolled_at DATETIME	Restrict access; encryption metadata and key version may be needed.
work_locations	id BIGINT; organization_id BIGINT; name VARCHAR; latitude DECIMAL; longitude DECIMAL; radius_m INT; active BOOLEAN	Latitude –90..90; longitude –180..180; radius positive.
attendance	id BIGINT; organization_id BIGINT; employee_id BIGINT; work_location_id BIGINT; clock_in_at DATETIME; clock_out_at DATETIME NULL; status VARCHAR	Indexes by tenant/employee/date; enforce one active shift according to business rule.
attendance_submissions	id BIGINT; organization_id BIGINT; department_id BIGINT; period_start DATE; period_end DATE; submitted_by BIGINT; status VARCHAR	Unique or conflict policy for department/period must be defined.
subscription_plans	id BIGINT; code VARCHAR; price_minor BIGINT; currency CHAR(3); active BOOLEAN	Store money as integer minor units (kobo) or DECIMAL consistently; currency NGN.
subscriptions	id BIGINT; organization_id BIGINT; plan_id BIGINT; status VARCHAR; trial_ends_at DATETIME NULL; current_period_end DATETIME NULL	FKs; state transition policy; prevent ambiguous active subscriptions.
payment_records	id BIGINT; organization_id BIGINT; subscription_id BIGINT; amount_minor BIGINT; currency CHAR(3); provider_reference VARCHAR; status VARCHAR	Unique provider reference; idempotent webhook processing.
support_tickets	id BIGINT; organization_id BIGINT NULL; created_by BIGINT; subject VARCHAR; status VARCHAR; priority VARCHAR	Ticket access must be scoped to owner/organization and Operations & Support.
audit_logs	id BIGINT; actor_user_id BIGINT NULL; organization_id BIGINT NULL; action VARCHAR; target_type VARCHAR; target_id VARCHAR; created_at DATETIME	Append-only by policy; index actor, tenant, action and time.
email_verifications / password_resets	id BIGINT; user_id BIGINT; token_hash CHAR(64); expires_at DATETIME; used_at DATETIME NULL	Store only token hashes; single use; expiry enforced server-side.

ER diagram

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
erDiagram
  ORGANIZATIONS ||--o{ USERS : contains
  ORGANIZATIONS ||--o{ DEPARTMENTS : has
  ORGANIZATIONS ||--o{ EMPLOYEES : employs
  DEPARTMENTS ||--o{ EMPLOYEES : groups
  EMPLOYEES ||--o| FACE_PROFILES : enrolled_as
  EMPLOYEES ||--o{ FACE_VERIFICATIONS : attempts
  EMPLOYEES ||--o{ ATTENDANCE : records
  ORGANIZATIONS ||--o{ WORK_LOCATIONS : configures
  ORGANIZATIONS ||--o{ SUBSCRIPTIONS : holds
  SUBSCRIPTION_PLANS ||--o{ SUBSCRIPTIONS : defines
  SUBSCRIPTIONS ||--o{ PAYMENT_RECORDS : billed_by
  PAYMENT_RECORDS ||--o| PAYMENT_RECEIPTS : produces
  SUPPORT_TICKETS ||--o{ SUPPORT_MESSAGES : contains
  USERS ||--o{ AUDIT_LOGS : performs
```

The diagram is conceptual. Confirm whether employees and login identities are one-to-one or separate in the actual schema.

Relationships and integrity

- Every organization-owned row should have a reliable tenant relationship, directly or through a validated parent.
- Use foreign keys for parent-child integrity where supported by the deployment database.

• Choose ON DELETE behavior deliberately. For audit, payments, and attendance, archival/soft deletion is usually safer than cascading deletion.

• Use transactions for multi-row attendance changes, payment finalization, and subscription state transitions.

• Use UTC consistently for DATETIME/TIMESTAMP values and convert for display in Africa/Lagos.

Indexes and why they exist

Index pattern	Reason
`users(email)` UNIQUE, if global email uniqueness is the rule	Fast login lookup and duplicate-account prevention.
`employees(organization_id, employee_number)` UNIQUE	Prevents duplicate employee identifiers within an organization.
`departments(organization_id, name)`	Efficient tenant-scoped department search; optionally unique.
`attendance(organization_id, employee_id, clock_in_at)`	Employee history and period reporting.
`attendance(organization_id, clock_in_at)`	Organization date-range reports.
`work_locations(organization_id, active)`	Load valid work locations for a tenant.
`payment_records(provider_reference)` UNIQUE	Prevents duplicate payment finalization for the same provider transaction.
`support_messages(ticket_id, created_at)`	Loads ticket conversation in order.
`audit_logs(organization_id, created_at)` and `(actor_user_id, created_at)`	Investigations and audit filtering.
`email_verifications(token_hash)` and `password_resets(token_hash)`	Fast token lookup; token hashes must still be expiry/used-state checked.

Sample queries

```
-- Employee attendance history. Bind values using PDO.
SELECT id, clock_in_at, clock_out_at, status
FROM attendance
WHERE organization_id = :organization_id
  AND employee_id = :employee_id
  AND clock_in_at >= :start_utc
  AND clock_in_at < :end_utc
ORDER BY clock_in_at DESC;

-- Active work locations for one organization.
SELECT id, name, latitude, longitude, radius_m
FROM work_locations
WHERE organization_id = :organization_id
  AND active = 1;

-- Payments that need review (example state names must match code).
SELECT id, organization_id, amount_minor, currency, status, created_at
FROM payment_records
WHERE status IN ('pending', 'requires_review')
ORDER BY created_at ASC;
```

5. docs/04-roles-and-permissions.md

Authorization must be enforced server-side on every route, action, API endpoint, export, and record lookup. UI hiding is only a usability measure.

Capability	System Admin	Ops & Support	Org Admin	Dept Head	Employee
Manage platform settings	Yes	No	No	No	No
Manage organizations	Yes	No	No	No	No
Manage plans/subscriptions platform-wide	Yes	No	Own subscription only	No	No
Handle support tickets	No	Yes	Create/view own org tickets per policy	Create/view permitted tickets	Create/view own tickets
Manage organization departments	No	No	Yes	Own department only	No
Manage employees	Platform support as explicitly authorized	No	All in own organization	Own department	Own profile only
Configure work locations	No	No	Yes	No	No
Clock in/out with face + GPS	No	No	If also an employee account, separately authorized	If also an employee account, separately authorized	Yes
Manual attendance	No	No	May review/manage per policy	Own department, reason required	No
Submit attendance period	No	No	Review organization submissions	Submit own department	No
View organization reports	Platform-wide audit/aggregate as authorized	Ticket context only	Own organization	Own department	Own history only
View audit logs	Platform-wide	No, unless ticket-specific audit is explicitly granted	Organization scope only if implemented	No	No
View biometric data	No routine raw access	No	Enrollment/verification status only by default	No raw data	No raw data

Role capabilities and boundaries

System Administrator

Manages platform organizations, subscription plans and states, platform staff, settings, audit logs, and platform health. Should not casually inspect raw biometric vectors or employee content; privileged access should be minimal and audited.

Operations & Support

Handles support tickets and threaded conversations only. Does not manage organizations, attendance, subscription plans, payment settings, employee records, or platform configuration unless a separately documented permission is added.

Organization Administrator

Administers a single organization: departments, employees, work locations, reports, and that organization’s subscription. Cannot access another organization’s data or platform-wide settings.

Department Head

Administers the assigned department's employees, permitted manual attendance, submissions, and department reports. Cannot change platform plans, manage other departments, or override security rules without a specific audited permission.

Employee

Uses their own employee portal to clock in/out with configured face and GPS checks and view their own attendance history. Cannot edit other employees' records, change location boundaries, or submit attendance on behalf of a department.

What each role cannot do and why

• Operations & Support cannot alter attendance or payments: support scope is ticket handling, reducing privilege abuse.

• Department Heads cannot view another department's employee records: scope follows assigned department membership.

• Employees cannot choose arbitrary organization IDs or work locations to bypass tenant/geofence rules.

• Organization Administrators cannot manage global plans or other tenants: platform administration is a separate trust boundary.

• System Administrator privileges must be audited; privileged status is not a reason to bypass logging.

Data visibility rules

Data	Visibility rule
Employee profile	Employee sees own profile; Department Head sees assigned department; Organization Administrator sees own organization; platform access only when operationally necessary and audited.
Attendance	Employee sees own history; Department Head sees assigned department; Organization Administrator sees own organization; System Administrator sees platform-level records only as required.
Face embeddings	Encrypted and unavailable through ordinary UI. Access should be restricted to the verification service.
GPS coordinates	Capture only what is necessary for attendance validation; expose to authorized reviewers only.
Payment data	Organization Administrator sees their organization's billing records; System Administrator manages platform-wide billing; support staff see only ticket details needed for support.
Support messages	Ticket participants and authorized Operations & Support staff according to ticket policy.
Audit logs	Platform administrators; organization-scoped views only if implemented and explicitly permitted.

6. docs/05-authentication.md

Login flow

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
sequenceDiagram
  participant U as User
  participant W as Web app
  participant L as Login limiter
  participant D as Database
  U->>W: Submit email + password + CSRF
  W->>L: Check account/IP window
  L-->>W: Allow or throttle
  W->>D: Fetch account by prepared query
  W->>W: Verify bcrypt hash and account status
  alt Valid credentials and verified account
    W->>W: Regenerate session ID; load role and tenant scope
    W-->>U: Redirect to authorized dashboard
  else Invalid / inactive / unverified
    W-->>U: Generic failure message
  end
```

Registration flow

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
flowchart TD
  A[Visitor submits organization registration] --> B[Validate fields and CSRF]
  B --> C[Check uniqueness and rate limits]
  C --> D[Create pending organization/account in transaction]
  D --> E[Generate random verification token]
  E --> F[Store SHA-256 token hash + expiry]
  F --> G[Send verification email]
  G --> H[User opens verification link]
  H --> I{Hash matches, unused, unexpired?}
  I -- Yes --> J[Mark verified and activate allowed state]
  I -- No --> K[Reject; offer safe resend flow]
```

Email verification

• Generate a cryptographically random token, send the raw token only in the email, and store only its SHA-256 hash.

• Use an expiry and single-use marker; invalidate older tokens on resend if that is the chosen policy.

• Do not disclose whether arbitrary email addresses already exist in public responses.

• Build links from a trusted configured HTTPS base URL, not the incoming Host header.

• Rate-limit verification resend requests and log delivery outcomes without logging the raw token.

Password reset

• Return a generic response regardless of whether the submitted email exists.

• Generate a high-entropy random token; persist only its SHA-256 hash with expiry and unused state.

• On reset, validate token, expiry and account state; update the password using bcrypt; mark the token used and invalidate other active reset tokens.

• Revoke or rotate existing sessions after password reset where supported.

• Never send passwords by email or store reset tokens in logs.

Session management

Control	Requirement
Session ID	Regenerate after successful login and privilege changes.
Cookie flags	Set `Secure` under HTTPS, `HttpOnly`, and an appropriate `SameSite` policy.
Transport	HTTPS only in production.
Idle/absolute timeout	Define and enforce explicit timeouts; do not leave sessions indefinite.
Logout	Invalidate server-side session and expire cookie.
CSRF	Validate a session-bound unpredictable token on every state-changing POST; protect equivalent state-changing endpoints too.
Role changes	Refresh/revoke sessions after role or account status changes.
Storage	Use secure server-side session storage; protect session directory/handler.

Rate limiting rules

The stated policy is five login attempts per 15 minutes. Define whether the counter is per account, IP, or a combination; a combined policy is more robust but must avoid easy denial of service against a victim account.

```
# Pseudocode for policy, not drop-in code:
if attempts(account_key, ip_key, last_15_minutes) >= 5:
    deny_login_temporarily()
else:
    verify_password()
    record_attempt(outcome)
```

Token security

• Use a cryptographically secure random generator for tokens.

• Store SHA-256 hashes of activation/reset tokens, not plaintext tokens.

• Compare token hashes safely, enforce expiry, and ensure single-use behavior transactionally.

• Use separate token purposes and do not reuse verification tokens as reset tokens.

• Protect signing/encryption keys outside source control; rotate them through a planned process.

7. docs/06-face-verification.md

The brief specifies 128-dimensional embeddings and possible browser ONNX, AWS, or Azure engines. It does not identify a specific model, service API, threshold, liveness detector, or current integration status. The deployment must document which engine is actually wired into the application.

Enrollment process

• An authorized administrator initiates employee enrollment after confirming identity under the organization’s process.

• The employee receives a clear notice describing purpose, retention, access, and the consequences of enrollment.

• The browser or configured provider captures an image under permission and connectivity constraints.

• The selected model detects/aligns a face and generates an embedding. A 128-dimensional vector means 128 numeric features; it is not itself a photograph or proof of identity.

• Validate model version, output dimension, quality and error states. Reject malformed or low-quality enrollment results.

• Encrypt the embedding before storage using authenticated encryption where available; store key identifiers separately from ciphertext.

• Record enrollment status, model version, consent/notice acknowledgement where applicable, and timestamp. Avoid retaining source images unless there is a documented need.

Verification process

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
sequenceDiagram
  participant E as Employee browser
  participant S as PHP server
  participant F as Face engine
  participant D as Database
  E->>E: Request camera permission and capture frame
  E->>F: Generate probe embedding (or send image to configured provider)
  F-->>E: Probe embedding / provider result
  E->>S: Submit attendance request + verification evidence
  S->>D: Load authorized employee's encrypted reference
  S->>S: Decrypt reference in controlled service
  S->>F: Compare embeddings / validate provider result
  F-->>S: Similarity score and engine metadata
  S->>S: Apply configured threshold and policy
  S-->>E: Accept or reject verification
```

If comparison occurs in the browser, the server must not blindly trust a client-supplied boolean such as `face_match=true`. A secure design either performs comparison in a trusted server/provider context or uses server-verifiable signed/provider results. Browser-only checks can be manipulated by a determined attacker.

Embedding generation and match threshold

A 128-dimensional embedding is an ordered vector of 128 numbers. Similarity is typically computed using a model-specific metric such as Euclidean distance or cosine similarity. The threshold depends on the exact model, preprocessing, camera conditions, and acceptable false-accept/false-reject trade-off.

```
# Conceptual only: do not use an arbitrary threshold in production.
distance = euclidean_distance(reference_embedding, probe_embedding)
matched = distance <= CONFIGURED_THRESHOLD
```

• Calibrate using representative consented test data and measure false acceptance and false rejection.

• Record model/version and threshold version for reproducibility.

• Do not assume scores are comparable across different models or providers.

• Fail closed if the face service is unavailable or returns malformed output, unless an explicitly approved fallback exists.

Encryption at rest

• Use PHP Sodium authenticated encryption where supported, with random nonces and versioned key management.

• Store ciphertext and necessary nonce/key-version metadata; never store the encryption key beside the encrypted embedding.

• Restrict decryption to the verification service. Do not return embeddings in ordinary API responses or admin pages.

• Backups containing encrypted biometric data need the same access controls as the live database.

• Define key rotation, deletion, retention, and breach-response procedures before production.

Available engine options

Engine	Potential model placement	Important caveat
Browser ONNX	Inference can run on a supported device/browser	A browser-side model is not a trusted security boundary; device support, model delivery, and tampering risks apply.
AWS	Remote managed face service, if specifically integrated	Requires provider credentials, network calls, region/privacy review, costs and provider-specific response validation.
Azure	Remote managed face service, if specifically integrated	Requires provider credentials, service configuration, region/privacy review, costs and provider-specific response validation.

These are engine choices stated in the brief, not confirmation that all three are implemented or selectable in the current build.

Privacy protections

• Provide clear notice and lawful basis/consent process appropriate to the jurisdiction and organizational context.

• Collect the minimum data needed; do not retain raw camera frames by default.

• Restrict biometric data access, encrypt storage and transport, log administrative access, and set retention/deletion rules.

• Provide a documented exception/review process for employees who cannot use face verification.

• Review Nigerian data protection requirements and conduct a privacy impact assessment before live deployment.

Honest limitations

Browser face matching alone does not establish that the camera is observing a live person. Without an implemented and tested liveness/anti-spoofing mechanism, a printed photograph or screen replay may fool some systems. Device-level spoofing, virtual cameras, manipulated JavaScript, and compromised browsers are also possible. Do not market the system as spoof-proof or liveness-verified unless those protections exist and have been independently tested.

8. docs/07-gps-geofencing.md

How geofencing works

An organization administrator configures a work location with latitude, longitude, and an allowed radius. At clock-in/out, the browser requests location permission and submits coordinates and accuracy metadata. The server calculates the distance from the configured location and applies the location policy. Browser coordinates are untrusted input; they are not cryptographic proof of physical presence.

Admin setup

• Choose a work-location point that represents the allowed attendance area.

• Enter valid latitude (–90 to 90) and longitude (–180 to 180).

• Set a radius in meters that reflects the real site and expected GPS error.

• Restrict who can create or edit locations and audit every change.

• Test both inside and outside the boundary on representative devices before enabling attendance.

Haversine calculation

The Haversine formula estimates great-circle distance between two latitude/longitude points on a sphere. Let ϕ be latitude in radians and λ longitude in radians:

```
a = sin²((ϕ2 - ϕ1) / 2)
    + cos(ϕ1) * cos(ϕ2) * sin²((λ2 - λ1) / 2)
c = 2 * atan2(sqrt(a), sqrt(1 - a))
distance_m = EarthRadius_m * c
accepted = distance_m <= allowed_radius_m
```

Use a consistent Earth-radius constant, validate numeric inputs, clamp floating-point `a` into [0,1] before square roots, and perform the authoritative calculation on the server.

Accuracy policy

Condition	Recommended policy to define
High accuracy value / poor fix	Reject or ask the employee to retry; define a maximum acceptable reported accuracy.
Distance close to radius boundary	Use a documented boundary buffer/retry policy rather than silently changing the radius.
Permission denied	Explain that attendance cannot proceed under a GPS-required policy and provide the approved desk/manual process.
Missing or invalid coordinates	Reject; never substitute a default location.
Multiple work locations	Validate against a location the employee is authorized to use, not every location in the database indiscriminately.

Server-side validation

```
# Conceptual server policy:
validate(latitude, longitude, accuracy_m)
assert employee_is_active(employee_id)
assert location_belongs_to_current_organization(location_id)
assert location_is_active(location_id)
distance_m = haversine(server_location, submitted_coordinates)
if accuracy_m is missing or accuracy_m > MAX_ACCURACY_M:
    reject("Location accuracy is insufficient")
if distance_m > allowed_radius_m:
    reject("Outside approved work location")
# Continue only after authentication, face, CSRF, duplicate and schedule checks.
```

Limitations

• GPS accuracy varies indoors, near tall buildings, and across device hardware.

• Operating-system or device-level location spoofing can falsify coordinates; browser geolocation does not prove physical presence.

- VPNs do not necessarily affect GPS coordinates, but a compromised device can manipulate location APIs.
- Mitigations include anomaly detection, audit trails, reasonable accuracy thresholds, device/risk signals where available, and supervised desk attendance. These reduce risk but do not eliminate spoofing.
- Do not collect continuous background location unless a separate, justified feature and privacy review explicitly require it.

9. docs/08-attendance-workflow.md

Clock-in sequence

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
sequenceDiagram
  participant E as Employee
  participant B as Browser
  participant S as Attendance service
  participant F as Face verifier
  participant G as Geofence validator
  participant D as Database
  E->>B: Press Clock In
  B->>B: Obtain camera and GPS evidence
  B->>S: POST clock-in + CSRF + evidence
  S->>S: Authenticate, authorize, validate CSRF
  S->>F: Verify face for authenticated employee
  F-->>S: Match result / failure
  S->>G: Validate coordinates against allowed location
  G-->>S: In range / out of range
  S->>D: Transaction: check duplicate, create attendance
  D-->>S: Saved attendance record
  S-->>B: Clock-in result and timestamp
```

Clock-out sequence

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
sequenceDiagram
  participant E as Employee
  participant B as Browser
  participant S as Attendance service
  participant F as Face verifier
  participant G as Geofence validator
  participant D as Database
  E->>B: Press Clock Out
  B->>S: POST clock-out + CSRF + evidence
  S->>S: Authenticate and validate request
  S->>F: Verify employee face
  F-->>S: Match result
  S->>G: Validate current location
  G-->>S: In range
  S->>D: Transaction: find open shift and close it
  D-->>S: Clock-out saved
  S->>S: Calculate worked duration and update status
  S-->>B: Clock-out timestamp and hours
```

Business rules

Rule	Expected behavior
Clock-in requires face + location	Reject if either required verification fails or cannot be completed.
Clock-out requires face + location	Apply the same verification policy unless a formally approved exception is configured.
Duplicate prevention	Use a transaction and database constraint/locking strategy to prevent concurrent duplicate open shifts.
Clock-out without open shift	Reject or route to an explicitly authorized correction workflow; do not silently create a completed shift.
Late detection	Compare accepted clock-in timestamp in organization timezone with the configured work schedule/grace period. The schedule source must be confirmed in implementation.
Worked hours	Calculate from stored UTC timestamps; handle missing clock-out, breaks, overnight shifts and corrections using documented rules.
Manual attendance	Department Head selects employee/date/time and provides a reason; log actor and changes.
Submission locking	Submitted records become read-only for normal department editing until the authorized review/unlock process.

Late detection

Store the accepted event timestamp in UTC. Convert it to the organization’s configured timezone (default Africa/Lagos) for schedule comparison. Define whether the grace period is applied before marking late, how holidays and shift schedules are loaded, and what happens when the organization has not configured a schedule. Do not hard-code a guessed start time.

Duplicate prevention

```
-- Illustrative transaction outline; table/column names must match actual schema.
START TRANSACTION;
SELECT id
FROM attendance
WHERE organization_id = :org_id
  AND employee_id = :employee_id
  AND clock_out_at IS NULL
FOR UPDATE;

-- If an open record exists, reject a second clock-in.
-- Otherwise insert the new record, after all required checks.
COMMIT;
```

A row lock alone may not protect against every first-row race. Consider a database-enforced design, transaction isolation strategy, or dedicated active-shift key that makes the rule unambiguous.

Manual attendance by department head

• Verify the actor is the department head assigned to the employee's department.

• Require a reason and validate the selected employee, date, and times.

• Record the original value, new value, actor, reason, and timestamp in the audit log.

• Prevent changes to locked/submitted periods unless the documented review workflow permits them.

• Do not represent manual attendance as face/GPS verified; mark its source clearly.

Attendance submission and locking

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
stateDiagram-v2
  [*] --> Draft
  Draft --> Submitted: Department Head submits period
  Submitted --> UnderReview: Authorized reviewer opens review
  UnderReview --> Approved: Review accepted
  UnderReview --> Rejected: Correction requested
  Rejected --> Draft: Reopen by authorized action
  Approved --> Locked: Finalize / lock
```

This is a suggested lifecycle to clarify behavior. Use only the states that the code actually supports. Record who submitted/reviewed/approved and when. Ensure rejected or reopened records retain an audit trail.

Reports and exports

• Scope reports to the actor's authorized tenant and department before filtering by date.

• Use parameterized date-range queries and limit export size to prevent resource exhaustion.

• Display timezone and reporting period on exports; clarify whether timestamps are local or UTC.

• Restrict exported personal data and secure generated files. Avoid public predictable download URLs.

• Reconcile report totals against source attendance records and document treatment of manual, incomplete, and corrected records.

10. docs/09-subscriptions-and-payments.md

Plan model

Plan	Model guidance	Must be configured, not assumed
Trial	Time-limited evaluation state	Duration, included features, limits, start/end behavior.
Starter	Entry paid tier	NGN price, billing interval, employee/location limits.
Business	Higher-capacity tier	Feature and usage limits, price and support scope.
Enterprise	Custom or highest tier	Whether price is fixed or quoted, limits and service terms.

No plan prices, trial length, or feature limits were supplied. Do not invent them in UI or billing code.

Trial lifecycle

Mermaid source (paste into a Mermaid-compatible Markdown viewer to render):

```
stateDiagram-v2
  [*] --> Trialing: Organization verified / trial granted
  Trialing --> Active: Valid payment or approved activation
  Trialing --> Expired: Trial end reached
  Active --> PastDue: Renewal payment failed / pending
  PastDue --> Active: Payment verified
  PastDue --> Suspended: Grace period ends
  Active --> Cancelled: Cancellation requested
  Suspended --> Active: Authorized reactivation
```

State names are conceptual; align them with the actual enum/status values. Define what happens to attendance access, data retention, exports, and read-only access when a subscription expires.

Paystack direct purchase flow

• Authenticated Organization Administrator chooses a plan and sees the exact NGN amount and billing terms.

• The server creates a pending payment/subscription request using server-owned price data; never trust an amount sent by the browser.

• The server initializes Paystack checkout with a unique reference and the organization’s verified billing identity.

• The browser completes checkout on Paystack and returns to the application. A return URL is not proof of payment.

• The server verifies the transaction with Paystack’s verification endpoint and validates reference, status, amount, currency, and expected account.

• The application finalizes payment and subscription in a database transaction, with idempotency so repeated callbacks do not double-activate.

• A signed webhook provides a second notification path. Verify its signature, reject invalid requests, and safely process duplicate deliveries.

• Record provider reference, amount, currency, result, timestamps and reconciliation outcome. Do not log secret keys or full sensitive payment data.

Webhook safety net

• Verify the webhook signature exactly as required by Paystack’s current documentation and configured secret.

• Compare the transaction reference with a pending local record; verify amount and currency on the server.

• Make webhook handling idempotent using a unique provider reference and a transaction-safe state transition.

• Return appropriate HTTP statuses; avoid lengthy work inside the webhook request where asynchronous processing is available.

• Do not activate a subscription solely because the browser returned to a success page.

• Run periodic reconciliation for pending payments and keep an audit trail of conflicting events.

Manual bank transfer

• Display the organization’s payment instructions and a unique payment reference, if implemented.

• Allow the organization to submit payment evidence only through a protected authenticated workflow.

• Keep the request pending until an authorized reviewer verifies the bank transaction independently.

• Record reviewer, decision, reason, amount, currency, reference and timestamp.

• Do not treat an uploaded screenshot as proof that funds were received.

Receipts

A receipt should include a unique receipt number, payer/organization, amount in NGN, payment date, payment reference, plan/period, and status. Generate it only after payment is verified or label it clearly as pending/acknowledgement if the business process requires one before settlement. Protect receipt URLs and prevent IDOR access.

Subscription states and transitions

State concept	Meaning / policy
Trialing	Trial access until configured expiry.
Active	Entitled to the plan’s enabled features.
Past due / pending	Payment or renewal requires attention; define any grace period.
Expired	Trial or paid period ended; apply documented access restrictions.
Suspended	Access limited due to policy, billing, or administrator action.
Cancelled	Renewal stopped or subscription ended; clarify access through paid period.

Use one authoritative subscription entitlement service for page access and API actions. UI-only plan checks are not sufficient.

11. docs/10-security.md

Security measures

Control	Implementation requirement / verification
PDO prepared statements	Bind all data values; whitelist dynamic identifiers such as sort columns.
Bcrypt password hashing	Use `password_hash()` and `password_verify()`; allow rehash upgrades.
CSRF protection	Validate session-bound tokens on all state-changing requests, including administrative actions.
Output escaping	Escape values for the correct output context (HTML, attributes, JavaScript, URL).
Server-side role enforcement	Authorize every endpoint and object; deny by default.
Tenant isolation	Apply tenant scoping to every query, export, upload, and background job.
Login rate limiting	Five attempts per 15 minutes as stated; define account/IP strategy and safe recovery.
Hashed single-use tokens	SHA-256 hashes for verification/reset tokens; expiry and used-state enforced.
Sodium encryption	Encrypt biometric data and secrets using appropriate authenticated encryption and key separation.
HTTPS	Require HTTPS in production; use secure cookies and trusted base URLs.
Audit logging	Log sensitive actions with actor, target, outcome and time, while excluding passwords/tokens/keys.
Backups	Encrypt and restrict backups; test restoration regularly.

Threat model

Threat	Mitigation	Residual risk
SQL injection	PDO parameter binding; query review; least-privilege DB account	Unsafe dynamic SQL can still introduce injection.
Cross-tenant data exposure	Tenant-scoped policy and queries; automated isolation tests	A missed query or background task can leak data.
Credential stuffing / brute force	Rate limiting, generic errors, strong password policy, monitoring	Distributed attempts and account lockout abuse remain possible.
Session theft	HTTPS, secure cookie flags, rotation, timeouts, XSS prevention	Compromised endpoint/browser remains a risk.
CSRF	Unpredictable token and origin/cookie controls	Misconfigured exceptions or alternate endpoints can bypass protections.
XSS	Contextual escaping, safe DOM APIs, input validation, CSP where feasible	Unsafe third-party scripts and template mistakes remain risks.
Biometric spoofing	Trusted verification design, liveness if actually implemented, monitoring	Browser-only matching without liveness is vulnerable to photo/replay attacks.
GPS spoofing	Server distance check, accuracy rules, audit/anomaly detection	OS-level spoofing cannot be eliminated by browser coordinates alone.
Forged payment callback	Server verification, webhook signature, amount/currency checks, idempotency	Provider outage or configuration errors require reconciliation.
Secret leakage	Environment secrets, restricted access, rotation, log redaction	Host compromise can expose runtime secrets.
Insider misuse	Least privilege, audit trails, separation of duties	Privileged insiders can still misuse access; review logs.
Data loss	Automated backups, restore drills, retention policy	Recovery point depends on backup frequency.

What is not protected automatically

• Face embeddings and GPS values do not prove a person is physically present without robust trusted capture, anti-spoofing and device controls.

• HTTPS does not protect a compromised phone, browser, administrator workstation, or hosting account.

• Encryption at rest is ineffective if keys are stored beside ciphertext or exposed in logs/source control.

• Prepared statements do not fix broken authorization or tenant isolation.

• CSRF protection does not prevent XSS; output escaping must match context.

• Paystack redirect success does not prove settlement; server verification and reconciliation are required.

• An audit log is not tamper-proof merely because it exists. Restrict modification rights and consider external/append-only retention for high assurance.

Secrets management

• Keep `.env`, provider keys, encryption keys and SMTP passwords out of Git and public directories.

• Use different keys for development, staging and production.

• Prefer hosting secret management or environment configuration; restrict read access to the deployment account.

• Rotate credentials after exposure, staff departure, suspected compromise, or provider policy changes.

• Never log raw passwords, reset tokens, biometric embeddings, Paystack secret keys, or SMTP credentials.

• Document backup and recovery of encryption keys separately from encrypted database backups.

Session security

Use HTTPS-only cookies with Secure, HttpOnly, and a deliberate SameSite policy. Regenerate session IDs after authentication and privilege changes, enforce idle and absolute expiration, invalidate server-side sessions on logout, and revoke sessions when accounts are disabled. Configure the PHP session handler and storage permissions for the hosting environment.

Audit logging

Log sensitive actions such as role changes, employee creation/deactivation, location changes, manual attendance, submission/reopening, subscription changes, payment verification, receipt generation, and security-setting changes. Include actor ID, organization scope, action, target, outcome, timestamp and request correlation metadata where useful. Redact secrets and avoid logging unnecessary face or precise-location data. Protect logs from ordinary application users and define retention and review responsibilities.

12. docs/11-troubleshooting.md

Database connection errors

Symptom	Likely cause	Fix
Application shows SQLSTATE, connection refused, access denied, or missing table.	Wrong host/port/credentials; MySQL stopped; database not imported; PHP PDO MySQL extension unavailable.	Start MySQL in XAMPP; confirm DB settings and grants; test credentials; enable PDO MySQL; import the actual schema; inspect logs without exposing passwords.

Login issues

Symptom	Likely cause	Fix
Correct-looking credentials fail, redirect loops, or login returns to the same page.	Account unverified/inactive; wrong password hash flow; session cookie/path issue; base URL mismatch; rate limit triggered.	Verify account status; use password_verify() for bcrypt hashes; clear stale cookies; check session storage and APP_URL; wait for rate-limit window or use approved recovery.

Email not sending

Symptom	Likely cause	Fix
Verification/reset email never arrives or SMTP error appears.	Wrong SMTP host/port/encryption; blocked outbound SMTP; invalid credentials; Gmail password used instead of an App Password where required; SPF/DKIM/deliverability issue.	Check email_logs and provider response; test SMTP credentials; for Gmail accounts with 2-Step Verification, create and use an App Password if eligible; use TLS/587 or provider-documented settings; check spam and sender-domain DNS.

Paystack errors

Symptom	Likely cause	Fix
Checkout cannot initialize, verification fails, or payment remains pending.	Test/live key mismatch; invalid amount/currency; callback treated as proof; wrong webhook secret; network or provider error.	Confirm environment keys match mode; send amount from server-side plan pricing; verify reference server-side; validate webhook signature; inspect provider dashboard; reconcile pending transaction before manual intervention.

Face model loading issues

Symptom	Likely cause	Fix
Camera works but model never loads, inference fails, or embedding dimensions differ.	Model files missing/blocked; incorrect path; unsupported browser/WebAssembly; CORS/MIME issue; model version mismatch; network restrictions.	Check browser console/network tab; verify model asset path and MIME/CORS; use supported browser; check model file integrity/version and expected vector dimension; do not bypass verification to make attendance pass.

Camera permission errors

Symptom	Likely cause	Fix
Browser says camera denied or no camera found.	Permission denied, camera used by another app, insecure HTTP context, OS privacy setting, or unsupported device.	Use HTTPS in production or localhost for local development; grant browser/OS permission; close apps using camera; check selected camera and browser compatibility.

GPS permission errors

Symptom	Likely cause	Fix
Location unavailable, timed out, or outside geofence unexpectedly.	Permission denied; indoor/poor GPS; stale coordinates; inaccurate reading; incorrect configured coordinates/radius; device spoofing or browser issue.	Enable location services; retry outdoors or near a window; inspect reported accuracy; verify work-location coordinates and radius; reject stale/invalid readings and follow approved manual-attendance procedure.

404s and redirect loops

Symptom	Likely cause	Fix
Routes return 404 or redirect repeatedly between login and dashboard.	Wrong document root/base URL; rewrite rules missing; session cookie path/domain mismatch; role redirect points to nonexistent route; HTTPS proxy configuration issue.	Confirm project URL and public root; inspect Apache rewrite configuration; check APP_URL and cookie settings; verify route exists and role dashboard mapping; review server logs.

CSRF token errors

Symptom	Likely cause	Fix
POST requests return forbidden or token-invalid errors.	Expired session, missing hidden token, token not refreshed, wrong cookie domain, form submitted across tabs after session rotation.	Ensure every state-changing form includes current CSRF token; refresh expired forms; validate same session; do not disable CSRF checks as a workaround.

Attendance duplicate/open shift

Symptom	Likely cause	Fix
Employee cannot clock in because an open shift exists, or concurrent requests create duplicates.	Previous clock-out missing; race condition; database constraint absent; manual record conflict.	Review the employee’s open attendance record; use an audited correction flow; enforce transactional duplicate checks and a database-backed concurrency rule.

Wrong worked hours or late status

Symptom	Likely cause	Fix
Displayed hours or late flags disagree with expectations.	Timezone conversion error; UTC/local values mixed; schedule/grace rules missing; overnight shift or break policy not defined.	Store timestamps in UTC; convert consistently for display and schedule comparisons; verify schedule configuration; test daylight/time-boundary assumptions and overnight rules.

Receipt inaccessible or wrong

Symptom	Likely cause	Fix
Receipt link exposes another organization's receipt or displays incorrect amount.	Missing ownership check; predictable public ID; wrong currency unit conversion; receipt created before verified payment.	Authorize every receipt request against tenant scope; use non-guessable references plus authorization; store NGN/kobo consistently; issue final receipts only after verified payment.

Permission denied / cross-tenant access

Symptom	Likely cause	Fix
Role cannot open a page or sees data unexpectedly.	Role mapping, organization linkage or department assignment incorrect; query lacks tenant predicate; stale session permissions.	Check assigned role and scope; refresh/revoke session after role changes; test server policy and tenant-scoped queries; treat unexpected access as a security incident.

500 error / blank page

Symptom	Likely cause	Fix
Request fails without useful browser details.	PHP fatal error, missing extension, bad config, file permissions, incompatible PHP version.	Check server/PHP error logs; enable detailed errors only in local development; verify PHP version/extensions and permissions; never expose stack traces in production.

Operational diagnostic commands

```
# PHP version
php -v

# PHP modules (confirm pdo_mysql, openssl, sodium where required)
php -m

# Check database connection using the configured application credentials.
mysql -h 127.0.0.1 -u attendance_app -p attendance_flow

# In local development only, inspect Apache/PHP logs in the XAMPP logs directory.
# Do not publish logs containing personal data, tokens, or credentials.
```

Incident handling checklist

- Preserve relevant timestamps, request IDs, audit entries and provider references.
- Do not paste passwords, API keys, reset links, raw face embeddings, or sensitive employee data into support tickets.
- Contain suspected tenant leakage or payment fraud promptly; disable affected credentials or endpoints as appropriate.
- Restore service only after verifying the cause and testing the relevant security control.
- Document root cause, scope, corrective action, and a regression test.

Appendix A — Release readiness checklist

Area	Release gate
Schema	Actual SQL/migrations match the table inventory; the 21-vs-25 discrepancy is resolved.
Authentication	Verification, reset, rate limiting, session rotation and logout tested.
Authorization	All five roles tested at both UI and direct endpoint level.
Tenancy	Cross-organization and cross-department isolation tests pass.
Face	Engine/model/version/threshold documented; failure behavior tested; liveness claims accurate.
GPS	Coordinate/radius policy tested; poor accuracy and permission denial handled.
Attendance	Clock-in/out, duplicates, missing clock-out, manual corrections, locking and reporting tested.
Payments	Paystack test flow, webhook signature, idempotency, reconciliation and manual review tested.
Security	HTTPS, CSRF, escaping, prepared statements, secrets and secure cookies checked.
Operations	Backups restored in a test, logs protected, support process documented, debug disabled.
Privacy	Biometric/GPS notice, retention, deletion, access controls and applicable legal review completed.

Appendix B — Documentation and implementation gaps to resolve

- Confirm the actual folder structure and update README paths accordingly.
- Replace the reference database column designs with the exact schema and migration history.
- Resolve the stated 21-table count versus the 25 named tables.
- Specify work schedules, grace periods, breaks, overnight shifts, and late-detection rules.
- Specify the actual face engine, model, similarity metric, calibrated threshold, liveness status, and fallback behavior.
- Specify GPS maximum accuracy, boundary tolerance, and approved exception process.
- Specify subscription prices, trial length, feature limits, grace periods, and exact status enums.
- Confirm Paystack API version, webhook verification method, and reconciliation process.
- Specify audit-log retention, backup schedule, recovery objectives, and privacy/retention policy.

Appendix C — Relative document links

Markdown file	PDF section
`README.md`	Section 1
`docs/01-installation.md`	Section 2
`docs/02-architecture.md`	Section 3
`docs/03-database.md`	Section 4
`docs/04-roles-and-permissions.md`	Section 5
`docs/05-authentication.md`	Section 6
`docs/06-face-verification.md`	Section 7
`docs/07-gps-geofencing.md`	Section 8
`docs/08-attendance-workflow.md`	Section 9
`docs/09-subscriptions-and-payments.md`	Section 10
`docs/10-security.md`	Section 11
`docs/11-troubleshooting.md`	Section 12